

CSpack Kernel and Scattering-Matrix Library

September 2026

1 Purpose

CSpack evaluates energy redistribution kernels for photon-electron Compton scattering and neutrino scattering on electrons and positrons in isotropic media. The library provides fixed-scatterer kernels, thermal averages over electron or positron distributions, kernel moments, compressed spline representations, and scattering matrices for repeated-scattering solvers.

The photon part follows Sarkar, Chluba and Lee (2019), the CSpack Compton kernel paper [1]. The neutrino extension follows Chluba, Cyr, Uwabo-Niibo and Yamaguchi (2026), where the isotropic neutrino-electron kernel is reduced from the full collision integral to a compact kernel form and added to CSpack [2].

CSpack is a library rather than a standalone evolution solver. The example program `CSpack_main.cpp` illustrates common calls, while `CSpack/CSpack.h` collects the public C++ headers. A small C interface is declared in `CSpack/CSpack_c.h` for integration weights and photon scattering matrix setup.

2 Variables and Normalization

The main routines use dimensionless variables:

Symbol	Code name	Meaning
ω_0, ω_1	<code>omega0, omega1</code>	Incoming photon or neutrino energy in units of $m_e c^2$.
ω, ω_3	<code>omega, omega3</code>	Outgoing photon or neutrino energy in units of $m_e c^2$.
p_0, p_2	<code>p0, p2</code>	Scatterer momentum in units of $m_e c$.
γ	<code>gamma_f(p)</code>	Electron or positron Lorentz factor $\sqrt{1 + p^2}$.
Θ	<code>theta, The</code>	Scatterer temperature $kT_e/(m_e c^2)$.
x	<code>xarr[i]</code>	Grid energy, usually ω/Θ_g .
μ_e	<code>mue</code>	Electron chemical potential parameter used by the FD routines.

The low-level photon kernels return the redistribution shape with the Thomson normalization factored out. The low-level neutrino kernels return the redistribution shape after factoring out the reference weak-scattering normalization used in the neutrino-kernel paper. The neutrino matrix wrapper adds the low-energy x_0^2 scaling and the optional process normalization through `sigma_norm`.

3 Building

The repository builds through the top-level `Makefile`, which delegates to `Makefile.in`.

```
make lib
make bin
```

`make lib` creates `libCSpack.a`. `make bin` also links the example program `CSpack_main`. The default `Makefile.in` expects GSL and Boost headers/libraries under the configured Homebrew paths and enables OpenMP through `g++`.

For external C++ code, include the umbrella header and link against the static library plus the same numerical dependencies:

```
#include "CSpack.h"
```

4 Low-Level Kernel Routines

The fixed-scatterer kernels all use the function-pointer type

```
typedef double (*kernel_ptr)(double, double, double);
```

The arguments are (`incoming_energy`, `scatterer_momentum`, `outgoing_energy`). This common signature lets photon and neutrino kernels be passed through the same moment, FD-average, output, and representation machinery.

4.1 Kinematic Helpers

`CSpack_functions` contains the shared kinematic layer used by both photon and neutrino kernels:

Function	Role
<code>gamma_f(p)</code> , <code>pfunc(gamma)</code>	Convert between momentum and Lorentz factor.
<code>gamma_sc(omega0, p0, omega)</code>	Final scatterer Lorentz factor after an energy transfer.
<code>pfunc_sc(omega0, p0, omega)</code>	Final scatterer momentum after an energy transfer.
<code>omegamin</code> , <code>omegacrit</code> , <code>omegamax</code>	Allowed outgoing-energy limits for fixed incoming energy and scatterer momentum.
<code>Get_zone(omega0, p0, omega)</code>	Branch selector for the analytic kernel expressions.
<code>lambda_p</code> , <code>lambda_m</code> , <code>kappa</code>	Invariants and auxiliary variables used inside the kernel expressions.
<code>mb_dist_func</code> , <code>mb_dist_norm</code> , <code>pbar</code>	Relativistic Maxwell-Boltzmann distribution helpers for thermal averaging.

These are useful for diagnostics and for checking support limits before evaluating a kernel. Most users call the higher-level kernel functions directly.

4.2 Photon Kernels

CSpack_kernels implements photon-electron redistribution kernels:

Function	Use
<code>kernel_exact(omega0, p0, omega)</code>	Exact isotropic Compton kernel for a fixed electron momentum.
<code>kernel_recoil</code>	Recoil-dominated limiting kernel.
<code>kernel_doppler</code>	Doppler-dominated limiting kernel.
<code>kernel_ur</code>	Ultra-relativistic limiting kernel.
<code>kernel_all(..., type)</code>	String-dispatched fixed-momentum photon kernel.
<code>Get_kernel_pointer(type, calling_func)</code>	Returns a <code>kernel_ptr</code> for <code>exact</code> , <code>recoil</code> , <code>doppler</code> , or <code>ur</code> .

The exact kernel first checks the allowed outgoing-energy interval, chooses the appropriate scattering zone, computes the branch variables, and evaluates the analytic expression. Outside the allowed kinematic region it returns zero.

4.3 Neutrino Kernels

CSpack_kernels_nu implements neutrino scattering kernels with the same `kernel_ptr` signature:

Function	Process
<code>kernel_exact_nue_e</code>	Electron-neutrino scattering on electrons.
<code>kernel_exact_nue_p</code>	Electron-neutrino scattering on positrons.
<code>kernel_exact_nue_ep</code>	Sum of electron and positron targets.
<code>kernel_exact_numu_e,</code> <code>kernel_exact_numu_p</code>	Muon-neutrino scattering on electrons or positrons.
<code>kernel_exact_nutau_e,</code> <code>kernel_exact_nutau_p</code>	Tau-neutrino scattering on electrons or positrons.
<code>Get_neutrino_kernel_pointer(type, calling_func)</code>	Returns the process-specific <code>kernel_ptr</code> .
<code>Get_neutrino_kernel_norm(type, calling_func)</code>	Coupling normalization relative to <code>nue_e</code> .

Internally, the neutrino routines evaluate the zone-dependent analytic integrals `I20`, `I_alpha`, and `I_beta` with chiral-coupling combinations `alpha_LR` and `beta_LR`. The electron/positron and flavor variants are built by changing the weak couplings rather than duplicating the kinematic machinery.

5 Thermal Kernel Routines

There are two thermal-averaging layers. The photon Maxwell-Boltzmann wrappers live in CSpack_kernels:

Function	Use
<code>thermal_kernel_exact(omega0, omega, theta)</code>	Thermal average of the exact photon kernel.
<code>thermal_kernel_recoil,</code> <code>thermal_kernel_doppler,</code> <code>thermal_kernel_ur</code>	Thermal averages of the limiting photon kernels.
<code>thermal_kernel_SS_K,</code> <code>thermal_kernel_SS_C</code> <code>thermal_kernel_exact_SS_C</code>	Sazonov-Sunyaev approximation kernels.
<code>thermal_kernel_all(..., type)</code>	Exact kernel with the <code>SS_C</code> low-energy replacement.
<code>Get_thermal_kernel_pointer(type, calling_func)</code>	String-dispatched thermal photon kernel. Returns a photon thermal-kernel pointer.

The generic Fermi-Dirac thermal average lives in `CSpack_kernels_FD`:

Function	Use
<code>norm_FD(theta, mue)</code>	Normalization for the FD scatterer distribution.
<code>thermal_kernel_FD(omega0, omega, theta, mue, K, add_FB)</code>	Thermal average of any fixed-momentum kernel <code>K</code> .
<code>output_thermal_kernel(...)</code>	Writes simple thermal-kernel tables for plotting.

`thermal_kernel_FD` is the bridge used for neutrino kernels, and it can also be used with the photon `kernel_exact` when a FD scatterer distribution is desired. The `add_FB` flag controls final-state factors:

<code>add_FB</code>	Factors applied
0	No final-state factor.
1	Final-state electron blocking.
2	Final-state electron and neutrino blocking.
3	Final-state electron blocking and photon stimulation.

6 Link to Representation Routines

The representation layer avoids evaluating expensive thermal kernels at every matrix entry. It samples a thermal kernel around each incoming energy and stores the result as splines in the logarithmic energy-transfer variable.

The generic representation interface is

```
typedef double (*thermal_kernel_ptr)(double omega0,
                                     double omega,
                                     double theta,
                                     void *p);
```

This is deliberately one level above `kernel_ptr`. A `kernel_ptr` describes a fixed-scatterer kernel. A `thermal_kernel_ptr` describes a thermally averaged kernel that can be sampled and interpolated by `Kernel_representation`.

Two adapters connect the low-level kernels to the representation machinery:

Adapter	Input parameter block	What it calls
thermal_kernel_photon_KR	Optional std::string *type	thermal_kernel_all(omega0, omega, theta, type)
thermal_kernel_neutrino_KR	Kernel_representation_nu_params *	thermal_kernel_FD(omega0, omega, theta, mue, K, add_FB)

For neutrinos, `Kernel_representation_nu_params` stores `mue`, `theta_g`, `sigma_norm`, the selected fixed-momentum kernel `K`, and `add_FB`. The adapter then multiplies the FD-averaged kernel by `sigma_norm*x0*x0`, where $x_0 = \omega_0/\Theta_g$.

The practical flow is:

```
fixed-scatterer kernel_ptr
-> thermal averaging
-> thermal_kernel_ptr adapter
-> Kernel_representation splines
-> bin-averaged scattering matrix
```

This is why the same matrix code can handle photons, neutrinos, or another thermal kernel with the same adapter signature.

7 Structograms

These Nassi-Shneiderman style diagrams summarize the main call paths.

Fixed-Momentum Kernel Evaluation
Input <code>omega0</code> , <code>p0</code> , <code>omega</code> , selected <code>kernel_ptr</code> <code>K</code> .
Compute kinematic limits with <code>omegamin</code> , <code>omegacrit</code> , <code>omegamax</code> .
If <code>omega</code> is outside the allowed interval, return zero.
Determine the analytic branch with <code>Get_zone</code> or the photon branch tests.
Evaluate the exact or approximate kernel expression.
Return the dimensionless redistribution kernel.

Thermal Averaging
Input <code>omega0</code> , <code>omega</code> , <code>theta</code> , distribution parameters, fixed kernel <code>K</code> .
Determine the lower momentum limit from the scattering kinematics.
Choose the upper integration range from <code>pbar(theta)</code> and the distribution tail.
Integrate over <code>log(p)</code> with Patterson quadrature.
Apply Maxwell-Boltzmann or Fermi-Dirac normalization.
Optionally multiply by final-state blocking or stimulation factors.
Return the thermally averaged kernel.

Kernel Representation and Matrix Setup
Input grid <code>xarr</code> , temperatures <code>theta</code> and <code>theta_g</code> , matrix threshold <code>epsilon</code> .
Select photon type or neutrino process and construct the thermal-kernel adapter.
For each incoming grid point, initialize <code>Kernel_representation</code> .
Sample the thermal kernel on the up/down logarithmic wings.
Store positive kernel values in splines and compute requested moments.
Integrate each outgoing bin explicitly or through the representation.
Drop matrix entries below <code>epsilon</code> and assemble <code>Msc</code> .

8 Scattering Matrices

`CSpack_scattering_matrix` builds redistribution matrices on an energy grid. The grid values are usually $x = \omega/\Theta_g$; for the simplest photon calls $\Theta_g = \Theta$.

Function	Use
<code>Integral_weights</code>	Computes quadrature weights for a supplied grid.
<code>compute_scattering_matrix(..., Int_wi, Msc, type, epsilon)</code>	Explicit photon matrix using a thermal kernel.
<code>compute_scattering_matrix(..., nK, Msc, KR, ...)</code>	Photon or generic matrix using <code>Kernel_representation</code> .
<code>compute_scattering_matrix_bin_averaged</code>	Photon bin-averaged matrix with $\Theta_g = \Theta$.
<code>compute_scattering_matrix_bin_averaged_I</code>	Generic or photon bin-averaged matrix with separate <code>theta</code> and <code>theta_g</code> .
<code>compute_scattering_matrix_neutrino_bin_averaged</code>	Neutrino bin-averaged matrix using FD-averaged kernels.
<code>compute_sigma_tot</code>	Sums matrix rows into total scattering rates.
<code>Msc_representation</code>	Sparse matrix wrapper for one temperature.
<code>Msc_representation_Te</code>	Temperature-table wrapper for repeated use over a temperature range.

The matrix entries are stored as bin-integrated redistribution probabilities with the thermal-kernel normalization and grid weights already applied. The `epsilon` threshold can be used to compress very small entries; values below about $1.0\text{e-}4$ are the usual practical regime when compression is desired.

9 Kernel Moments and Fokker-Planck Coefficients

`CSpack_kernel_moments` exposes analytical and numerical moment routines:

Function	Use
<code>compute_exact_moments_analytical</code>	Fixed-electron exact Compton moments.
<code>compute_nr_moments_analytical</code> , <code>compute_ur_moments_analytical</code>	Non-relativistic and ultra-relativistic limits.
<code>compute_recoil_moments_analytical</code> , <code>compute_doppler_moments_analytical</code>	Limiting recoil and Doppler moments.
<code>moment_Int_therm</code> , <code>moment_Int_app1</code> , <code>moment_Int_app2</code> , <code>moment_Int_app3</code>	Thermal moment approximations.
<code>moment_2D_Int_therm_all_II</code>	Thermally averaged moment from a two-dimensional integration.
<code>compute_all_FP_coefficients</code>	Coefficients for a Fokker-Planck approximation.
<code>G_moment_2D_Int_therm_all_II</code>	Combination 2 <code>Sigma2</code> - <code>Sigma1</code> used by the representation layer.
<code>CSpack_kernel_moments_numerical::Sigma1</code> , <code>CSpack_kernel_moments_numerical::Sigma2</code>	Numerical fixed-scatterer moment for any <code>kernel_ptr</code> .

These routines are the connection between the full kernel description and diffusion-style approximations such as Kompaneets or related Fokker-Planck limits.

10 Other Public Helpers

Module	Main role
CSpack_weights	Builds trapezoidal, higher-order, log-grid, and weighted Lagrange integration weights.
CSpack_Collision_Terms	Evaluates collision terms and thermal moments for equilibrium input distributions.
CSpack_opacity	Computes generalized neutrino scattering optical depths and unity-crossing scales.
CSpack_energy_losses	Computes photon and electron energy exchange, removal, and double-Compton related helper terms.
CSpack_equilibrium_solutions	Converts between small temperature, number, energy, and chemical-potential perturbations.
CSpack_c.h	Provides C-callable Planck, integration-weight, and photon matrix setup wrappers.

11 Minimal Examples

Fixed-momentum photon and neutrino kernels:

```
#include "CSpack.h"

using namespace CSpack_kernels;
using namespace CSpack_kernels_nu;

double omega0 = 0.1;
double p0 = 0.2;
double omega = 0.12;

double Pph = kernel_exact(omega0, p0, omega);
double Pnu = kernel_exact_nue_e(omega0, p0, omega);
```

Simple output tables for plotting:

```
vector<double> p0a = {omega0/10.0, omega0/2.0, omega0, 2.0*omega0,
                    5.0*omega0};

output_kernel("./outputs/kernel_ph.dat", 2000, omega0, p0a,
             CSpack_kernels::kernel_exact);
output_kernel("./outputs/kernel_nu.dat", 2000, omega0, p0a,
             CSpack_kernels_nu::kernel_exact_nue_e);
```

Thermally averaged FD kernels:

```
using namespace CSpack_kernels_FD;

vector<double> omega1 = {0.001, 0.01, 0.1, 1.0, 10.0};
double theta = 0.1;
double mue = -1.0e+2;

output_thermal_kernel("./outputs/kernel_nu_FD.dat", 500, omega1, theta, mue,
```

```

        CSpack_kernels_nu::kernel_exact_nue_e, 0);
output_thermal_kernel("./outputs/kernel_nu_FD_block.dat", 500, omega1, theta,
        mue, CSpack_kernels_nu::kernel_exact_nue_e, 2);

```

Photon scattering matrix:

```

using namespace CSpack_scattering_matrix;
using namespace CSpack_weights;

vector<double> xarr;
init_xarr(1.0e-4, 1.0e+4, xarr, 300, 1, 0);

vector<double> Int_wi;
Integral_weights(xarr, Int_wi);

vector<vector<double>> Msc;
compute_scattering_matrix(xarr, theta, Int_wi, Msc, "exact+SS_C", 1.0e-8);

```

Neutrino bin-averaged matrix through the representation layer:

```

vector<vector<double>> Mnu;
vector<Kernel_representation> KR;

double theta_g = theta;
double sigma_norm = 1.0;

compute_scattering_matrix_neutrino_bin_averaged_II(
    xarr, theta, theta_g, Mnu, KR,
    "nue_ep", 0.0, 2, 1.0e-8, sigma_norm
);

```

12 Illustrative Kernel Plots

The repository already contains CSpack-generated figures under `plots/nu-scattering`. The figures below show a few simple kernel examples.

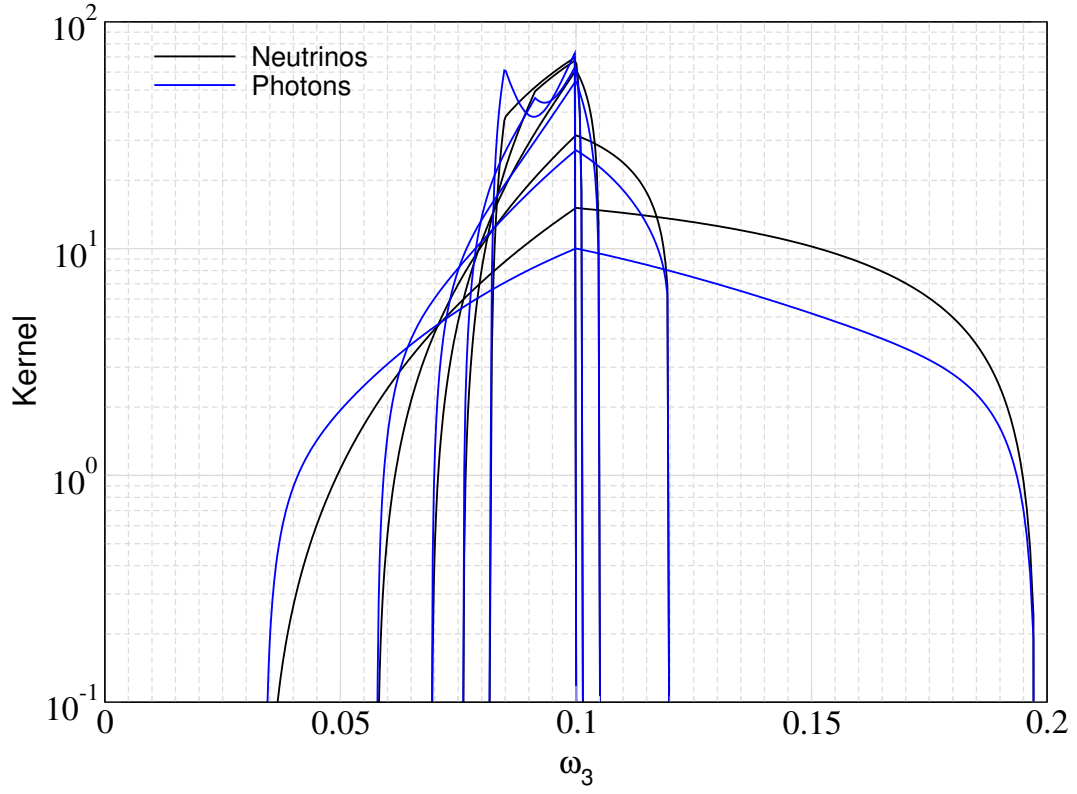


Figure 1: Fixed-momentum photon and electron-neutrino kernels for the same incoming energy and selected scatterer momenta.

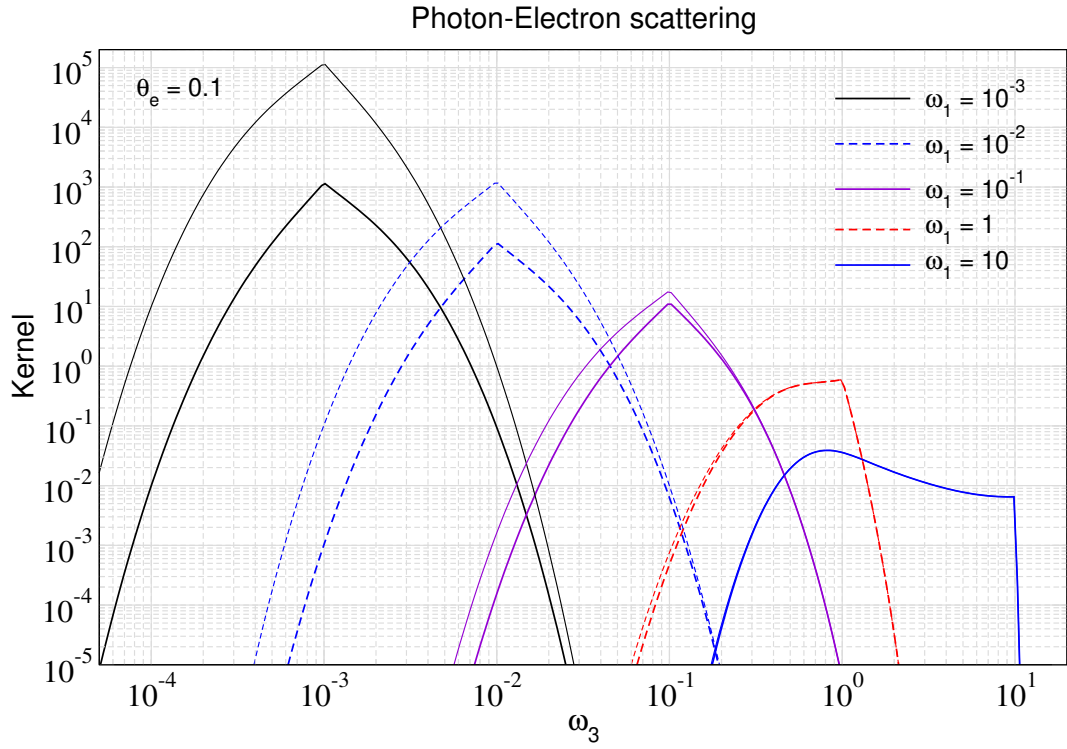


Figure 2: Thermally averaged photon-electron kernels at $\Theta_e = 0.1$ for several incoming energies.

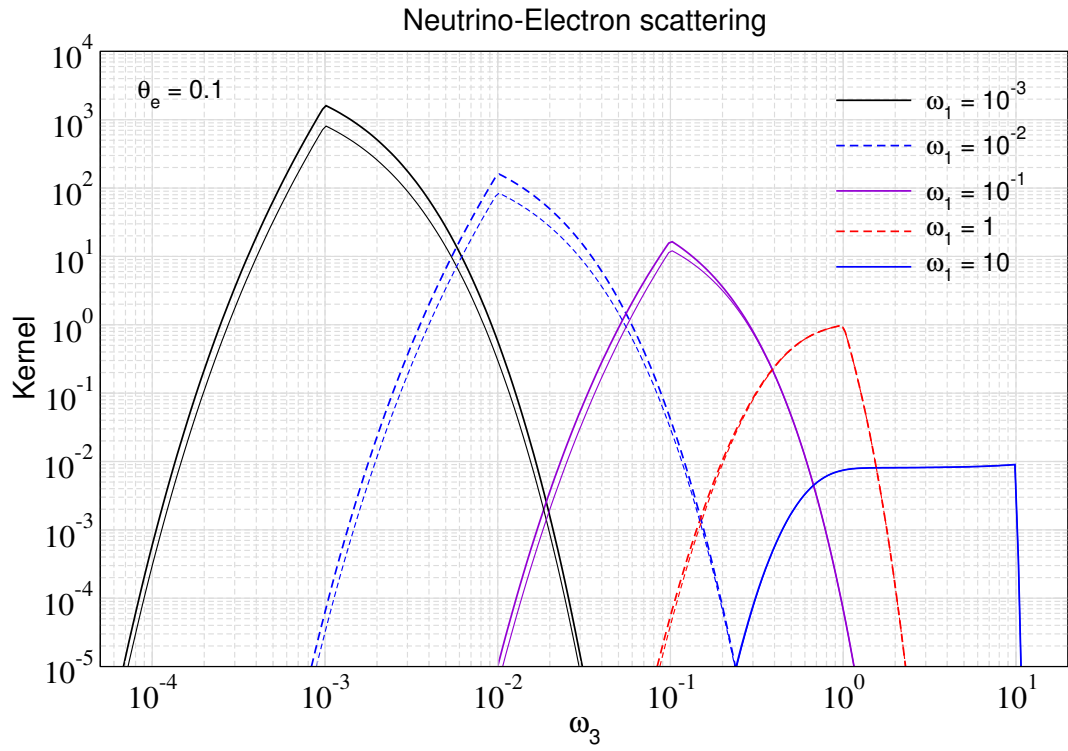


Figure 3: Thermally averaged electron-neutrino kernels at $\Theta_e = 0.1$ for the same incoming-energy sequence.

13 References

References

- [1] A. Sarkar, J. Chluba and E. Lee, “Dissecting the Compton scattering kernel I: Isotropic media,” *Monthly Notices of the Royal Astronomical Society* **490**, 3705–3726 (2019), arXiv:1905.00868. <https://arxiv.org/abs/1905.00868>
- [2] J. Chluba, B. Cyr, M. Uwabo-Niibo and M. Yamaguchi, “Neutrino-electron scattering kernels in isotropic media,” arXiv:2607.25436. <https://arxiv.org/abs/2607.25436>
- [3] S. Y. Sazonov and R. A. Sunyaev, “Cosmic microwave background radiation in the direction of a moving cluster of galaxies with hot gas: relativistic corrections,” *Astrophysical Journal* **543**, 28–55 (2000).